NEXUS AI Research Lab | Audited & Verified: 150 Benchmark Episodes | Multi-Agent RL & Simulation

STRATOS AI: Strategic Team Reasoning & Tactical Operations

Comprehensive Internal Project Review & Tutor Defense Master Guide

---

⚡ 60-Second "Elevator Pitch" for Tomorrow's Review

(Give this exact 1-minute answer when tutors ask: "What is your project about and what have you accomplished?")

> "Our project is **STRATOS AI**, an autonomous multi-agent reinforcement learning system designed to solve a critical open problem in AI: **Does real-time communication between specialized AI agents improve their coordination and tactical performance against complex opponents compared to agents operating independently?**
>
> We built a custom top-down real-time strategy environment adhering to **PettingZoo Parallel API** standards. On this battlefield, a squad of three specialized friendly agents (a **Commander**, a **Defense Guard**, and an **Attack Striker**) collaborate to defend their base, harvest crystals, and defeat hostile squads.
>
> The agents are governed by a deep Actor-Critic policy trained using **Proximal Policy Optimization (PPO)** coupled with an inter-agent **Tactical Communication Bus**. In our multi-seed benchmark of 150 episodes across 5 seeds, coordinated agents achieved a **100% win rate** compared to **66.7%** for independent agents (+33.3% communication advantage). Most notably, against aggressive rush attacks, independent agents suffered a 100% wipeout loss (0% win rate), while coordinated agents communicated base-defense directives to win 100% of matches."

🎮 1. The Game Concept & Battlefield Mechanics

STRATOS is inspired by classic RTS games (like *StarCraft*), designed as a mathematical MARL testbed:

- **Arena Size**: 800 × 600 continuous 2D coordinates with bounded edges.
- **Friendly HQ**: (80, 300) with 1,000 HP. If lost, friendly team loses.
- **Enemy HQ**: (720, 300) with 1,000 HP. If destroyed, friendly team wins.

- **Crystal Depots (Resources)**: Three contested mining nodes at North (400, 150), Center (400, 300), and South (400, 450). Units mine resources by positioning over them.
- **Squad Composition**:
  - **Commander** (280 HP, 3.2 speed): Strategic priority issuer & resource manager.
  - **Defense Guard** (380 HP tank, 2.6 speed): High armor base anchor, shields the HQ.
  - **Attack Striker** (220 HP scout, 3.8 speed, 32 dmg): High-damage flanker & siege specialist.
- **Three Opponent Types**:
  1. **Aggressive Rush**: Directly attacks friendly HQ with no resource harvesting.
  2. **Defensive Turtle**: Tightly guards the enemy base perimeter (150px radius).
  3. **Resource-Focused**: Mines central crystal deposits to 100 units before transitioning to an assault.

🛠️ 2. The Complete Technical Stack

Component	Technology	Role in STRATOS
Core Language	Python 3.11+	Primary environment, agent logic, and simulation engine.
Deep Learning	PyTorch (`torch.nn`, `Adam`)	Actor-Critic Policy network, PPO loss computation, backpropagation.
MARL Standard	PettingZoo (Parallel API) & Gymnasium	Multi-agent step execution, decentralized observations, individual rewards.
RL Algorithm	PPO + GAE-λ	On-policy Actor-Critic with clipped surrogate loss and value estimation.
Visualization	Pygame	Real-time 60 FPS battlefield canvas, lasers, HUD telemetry, live radio ticker.
Analytics UI	Streamlit, Pandas, Matplotlib	Interactive research dashboard showing win rates, learning curves, and distributions.
Quality Assurance	Pytest	20 automated unit & integration tests passing at 100%.

🏛️ 3. System Architecture & Information Pipeline

The 49-Dimensional Observation Vector

Every step, each agent receives a normalized vector in `[-1.0, 1.0]`:

- **Channels 0–4 (Self State)**: (x_{norm}, y_{norm}, HP ratio, cooldown ratio, alive flag)
- **Channels 5–7 (Friendly HQ)**: (x_{norm}, y_{norm}, HP ratio)
- **Channels 8–10 (Enemy HQ)**: (x_{norm}, y_{norm}, HP ratio)
- **Channels 11–18 (2 Teammates)**: 2 × (Δx, Δy, HP ratio, alive flag)
- **Channels 19–30 (3 Enemies)**: 3 × (Δx, Δy, HP ratio, alive flag)
- **Channels 31–39 (3 Resource Nodes)**: 3 × (Δx, Δy, crystal ratio)
- **Channels 40–42 (Global State)**: Friendly crystals, enemy crystals, episode step ratio
- **Channels 43–48 (Tactical Comm Bus)**: Directive code, threat level, target Δx, target Δy, support flag, message priority

The 7 Discrete Macro-Actions

`HOLD (0)` • `MOVE_BASE (1)` • `MOVE_ENEMY_BASE (2)` • `MOVE_RESOURCE (3)` • `ATTACK_NEAREST_ENEMY (4)` • `DEFEND_PERIMETER (5)` • `TACTICAL_RETREAT (6)`

📡 4. The Inter-Agent Communication Bus & 15-Step Decay

STRATOS uses a **structured symbolic messaging bus** rather than black-box latent vectors:

- **Messages Supported**: `ENEMY_DETECTED`, `DEFEND_BASE`, `REQUEST_SUPPORT`, `ATTACK_TARGET`, `RESOURCE_LOW`, `RESUME_OFFENSE`

- **Why Symbolic?** 100% human-interpretable in real time, low bandwidth, and zero black-box obscurity.
- **15-Step Temporal Freshness Decay**: In early builds, emergency support alerts stayed in inboxes forever, locking units into defensive loops. We engineered a decay filter where messages older than 15 steps are discarded.

📊 5. Empirical Benchmark Results (150 Episodes Across 5 Seeds)

Scenario / Opponent	Condition	Win Rate	Mean Reward	Survival Rate	Avg Messages
Aggressive Rush	**Independent**	**0 / 25 (0.0%)**	-408.89	0.0% (Squad Wipeout)	0.0 msgs
Aggressive Rush	**Coordinated**	**25 / 25 (100.0%)**	**+242.26**	**33.3%**	**819.0 msgs**
Defensive Turtle	Independent	25 / 25 (100.0%)	+556.86	100.0%	0.0 msgs
Defensive Turtle	Coordinated	25 / 25 (100.0%)	+556.86	100.0%	697.0 msgs
Resource Focused	Independent	25 / 25 (100.0%)	+408.82	33.3%	0.0 msgs
Resource Focused	Coordinated	25 / 25 (100.0%)	+408.82	33.3%	503.0 msgs

Key Benchmark Takeaways

- **Overall Win Rate**: Independent = 66.7% vs **Coordinated = 100.0% (+33.3% net advantage)**.
- **Mean Team Reward**: +185.60 vs **+402.65 (+217.05 points higher)**.
- **The Aggressive Rush Breakthrough**: Independent agents get wiped out 100% of the time (0% win rate). Coordinated agents broadcast `DEFEND_BASE`, assemble perimeter defenses, and achieve a **100% win rate (+100.0% turnaround)**.

💻 6. How to Run the Live Demos for the Tutors

1. Real-Time Pygame Demonstration

```
.\.venv\Scripts\python scripts/run_demo.py
```

Interactive Keys to Show Off:

- `[M]`: **Live Mode Toggle** — switches between Coordinated and Independent mode on the fly! Notice the bottom radio ticker silence and units disperse.
- `[SPACE]`: Pause and resume simulation.
- `[UP / DOWN]`: Speed up or slow down simulation FPS.
- `[R]`: Reset episode.

2. Interactive Research Analytics Dashboard

```
.\.venv\Scripts\streamlit run dashboard/app.py
```

Launches a web dashboard displaying side-by-side win rate charts, learning curves, and scenario breakdowns.

3. Run Automated Test Suite

```
.\.venv\Scripts\pytest -q
```

Executes all 20 regression tests (asserts 100% pass rate in ~11 seconds).

🛡️ 7. Tutor Q&A Defense (The Top 6 Questions & Model Answers)

Q1: "Did you just hardcode rules, or is this genuine Reinforcement Learning?"

Answer: "It is genuine Reinforcement Learning. The agents' action selection is governed by an Actor-Critic Neural Network trained via Proximal Policy Optimization (PPO) in PyTorch. The policy receives a 49-dimensional continuous observation vector and outputs probability distributions over 7 actions. What is structured is the communication schema, which feeds directly into observation channels 43 to 48. The policy network learned through PPO updates when to react to those messages."

Q2: "How do you prove the policy actually uses the communication channels rather than ignoring them?"

Answer: "We conducted a formal Policy Sensitivity Audit on the neural network weights. By keeping physical observations (channels 0–42) completely identical and only modifying the communication channels (43–48), we observed a measurable shift in action logits (Δ_{max} = 0.0516) and a +0.931 shift in the Critic's state-value estimation. Furthermore, in our 150-episode benchmark, zeroing out the comm channels caused a 100% squad wipeout against aggressive rushers, while active comm channels delivered a 100% win rate."

Q3: "Why PettingZoo instead of standard Gym / Gymnasium?"

Answer: "Gymnasium is designed for single-agent systems. PettingZoo is the multi-agent standard, specifically the Parallel API, which allows simultaneous agent execution, decentralized observations, and individualized reward vectors."

Q4: "Why PPO over DQN or SAC?"

Answer: "DQN struggles with non-stationary multi-agent dynamics because replay buffers contain experiences generated by earlier teammate policies. SAC is designed for continuous action spaces, whereas our macro-actions are discrete. PPO with GAE is on-policy, stable, resilient to hyperparameter variations, and avoids destructive updates via clipped surrogate objectives."

Q5: "What real engineering challenges did you face?"

Answer: "Two major bugs: First, stale inbox message persistence, where support alerts stayed active forever; we solved this by designing a 15-step temporal freshness decay filter. Second, broken command succession when the Commander died; we implemented dynamic command succession so the Defense Guard immediately assumes tactical leadership."

Q6: "What is your future roadmap?"

Answer: "Scaling training to 30,000+ timesteps to compress policy entropy, introducing Asymmetric Self-Play where opponents co-evolve, and benchmarking continuous latent communication (such as TarMAC or CommNet) against our symbolic protocol."
